

Kennesaw State University
CS8267 – Advanced Machine Learning
Fall 2026

Assignment 1

Agentic Multi-Document Summarization Using Clustering and Large Language Models

***** NOTE:** You cannot paste the assignment partially or wholly to any AI model. However, you may use permissible AI models to complete the assignment.

Purpose

Traditional summarization systems often rely on a single Large Language Model (LLM) to generate a summary directly from a collection of documents. However, modern AI systems increasingly use **agentic workflows**, where multiple specialized agents collaborate to solve a complex task.

In this assignment, you will build an **agent-based summarization system** that combines machine learning techniques with Large Language Models. Instead of directly generating a summary, your system will discover major topics using **k-means clustering** and use those topics as building blocks for the final summary.

You will implement several cooperating agents that process documents, generate embeddings, identify topics through clustering, create summary components, and finally assemble a coherent summary.

This assignment introduces the concepts of:

- Agent orchestration
- Retrieval-Augmented Generation (RAG)
- Text embeddings
- Topic discovery through clustering
- Multi-document summarization
- Large Language Models

Learning Goals

After completing this assignment, students should be able to:

- Understand agent-based AI architectures.
- Understand Retrieval-Augmented Generation (RAG) fundamentals.
- Generate and use text embeddings.
- Learn how distance/similarity based clustering algorithms work
- Implement the k-means clustering algorithm from scratch.
- Apply clustering to document analysis and summarization.
- Use Large Language Models for topic extraction and paraphrasing.
- Understand how to get embeddings for text
- Design and orchestrate multiple AI agents.
- Evaluate the strengths and weaknesses of cluster-based summarization versus direct LLM summarization.

Description

You will build an **Agentic Multi-Document Summarization System** that analyzes a collection of related documents and creates a summary through a sequence of collaborating agents.

Clustering is an unsupervised method for grouping patterns based on similarity such that the patterns in the same group are similar to one another, and those in different groups are less similar to one another. Clustering has many uses, including outlier detection and unsupervised image segmentation. The K-Means clustering algorithm is one of the most widely used clustering algorithms. In this assignment, you are going to implement basic kmeans.

Unlike traditional summarization approaches, your system should discover major topics within the documents using clustering. Each cluster should represent a topic or concept discussed across the document collection.

The representative text associated with each cluster will become a summary component. A final agent will organize these components into a coherent summary.

Data Collection

Select a topic related to machine learning, artificial intelligence, data science, cybersecurity, or computer science. Choose **three closely related documents** manually.

Examples include:

- Three articles about transformers
- Three articles about large language models
- Three articles about retrieval-augmented generation
- Three articles about cybersecurity threats
- Three research papers addressing a similar topic

Documents may be:

- PDF files
- TXT files
- HTML files
- Local copies of web articles
- Research papers

All documents should be downloaded and stored locally before processing.

Part 1: Verify Your K-Means Implementation

Before applying clustering to textual data, implement and verify the correctness of “**your**” k-means algorithm from scratch using the provided:

- kmtest.arff
- or kmtest.csv

Requirements:

- Implement k-means from scratch.
- Do not use existing k-means implementations from machine learning libraries.
- Ignore class labels if they exist.
- Report cluster assignments, cluster centroids, and the number of iterations until convergence

Part 2: Build an Agentic Workflow

Your system should contain at least the following agents.

Agent 1: Document Processing Agent

Responsibilities:

- Load the three documents.
- Extract text.
- Split text into overlapping chunks.

You may use:

- LangChain document loaders
- RecursiveCharacterTextSplitter
- Similar chunking utilities

Output:

Document Chunks

Agent 2: Embedding Agent

Responsibilities:

- Generate embeddings for all chunks.
- Store embeddings if desired (e.g., ChromaDB).
- Return embedding vectors for clustering.

You may use:

- HuggingFace Embeddings
- Sentence Transformers
- Ollama Embeddings
- OpenAI Embeddings
- Other available embedding models

Output:

Chunk Embeddings

Agent 3: Clustering Agent

Responsibilities:

- Apply your own implementation of k-means.
- Cluster embedding vectors.
- Compute cluster centroids.
- Identify a representative chunk (medoid) for each cluster.

The medoid is the chunk whose embedding is closest to the cluster centroid.

Perform experiments using:

$k = 3$

$k = 10$

$k = 30$

Output clusters, centroids, and medoids.

Agent 4: Topic Extraction Agent

Responsibilities: For each cluster, obtain the medoid chunk. Use an LLM to generate a concise summary statement.

Output:

Summary Components

Agent 5: Summary Organization Agent

Responsibilities: Analyze all summary components, determine a logical ordering, and produce a coherent final summary.

Final Output:

Complete Multi-Document Summary

Agent Orchestration

Your agents should operate as a pipeline similar to:

[Document Processing] → [Embedding Agent] → [Clustering Agent]
→ [Topic Extraction] → [Summary Organization] → Final Summary

You may implement orchestration using:

- LangChain
- LangGraph
- CrewAI
- AutoGen
- Custom Python coordination logic

Experimental Analysis

Note that you have to make your program generate meaningful results. Improve your program based on the issues you observe.

Perform experiments using:

$k = 3$

$k = 10$

$k = 30$

For each value of k :

1. Report cluster sizes.
2. Report cluster medoids.
3. Report generated topic labels.
4. Present the final summary.

Discuss:

- Which value of k produced the best summary?
- Which value of k produced excessive redundancy?
- Which value of k produced insufficient topic coverage?
- How did cluster granularity affect summary quality?

Discussion Questions

1. How well did clustering identify major topics?

2. Were some clusters redundant?
3. Did increasing k improve or degrade summary quality?
4. Were the generated topic labels accurate?
5. What challenges arose when ordering summary components?
6. How effective was the agent-based workflow?

Deliverables

Submit:

1. Source code.
2. Three selected documents.
3. Brief description of the chosen topic.
4. Results for:
 - $k = 3$
 - $k = 10$
 - $k = 30$
5. Final generated summaries.
6. Discussion and analysis.
7. (Optional) Bonus comparison report.

Deliverables

- Submit your Jupyter notebook or similar with the outputs of all cells. Submission without outputs will not be graded.
- Make sure that cluster centers are computed and presented for the kmtest file.
- **Note about filename notation:** Your notebook file must start with your login name. Your file name will look like raygunClustering.ipnyb
- Submission by Email is not acceptable.
- All submissions are due class time unless stated otherwise.

Notes

The following additional comments apply:

1. If the requested work is a core component of the assignment, you cannot use the available libraries to run the assignment.
2. Good programming style must be observed. This includes using meaningful variable names, descriptive comments, and readable formatting
3. The program source code file must contain a header comment at the beginning of the file. This comment must include the following items:

- Source code file name
 - Student Name
 - Date
4. Students should work independently. Each student is responsible for handing in an original program.